

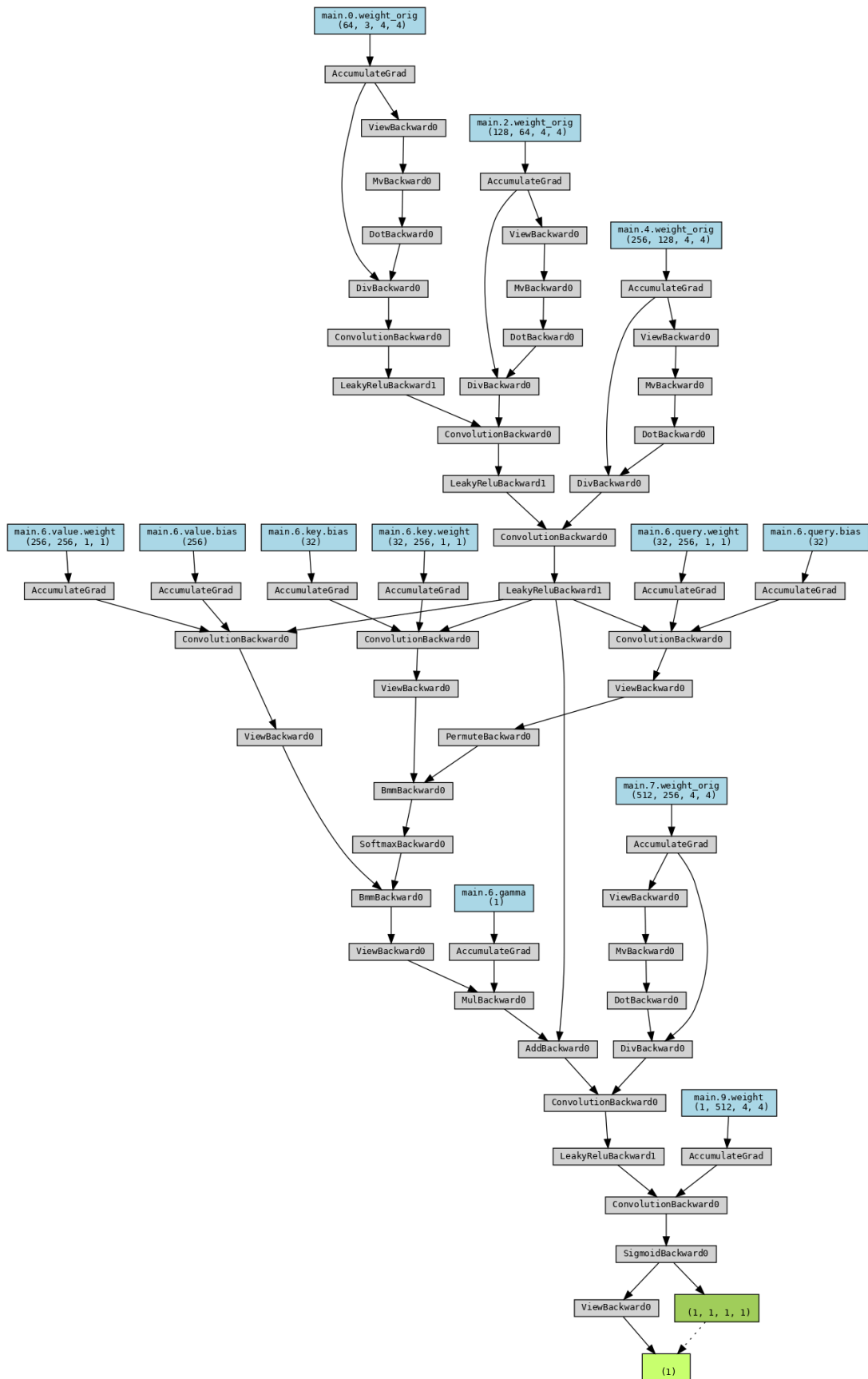
HW9

Execution description

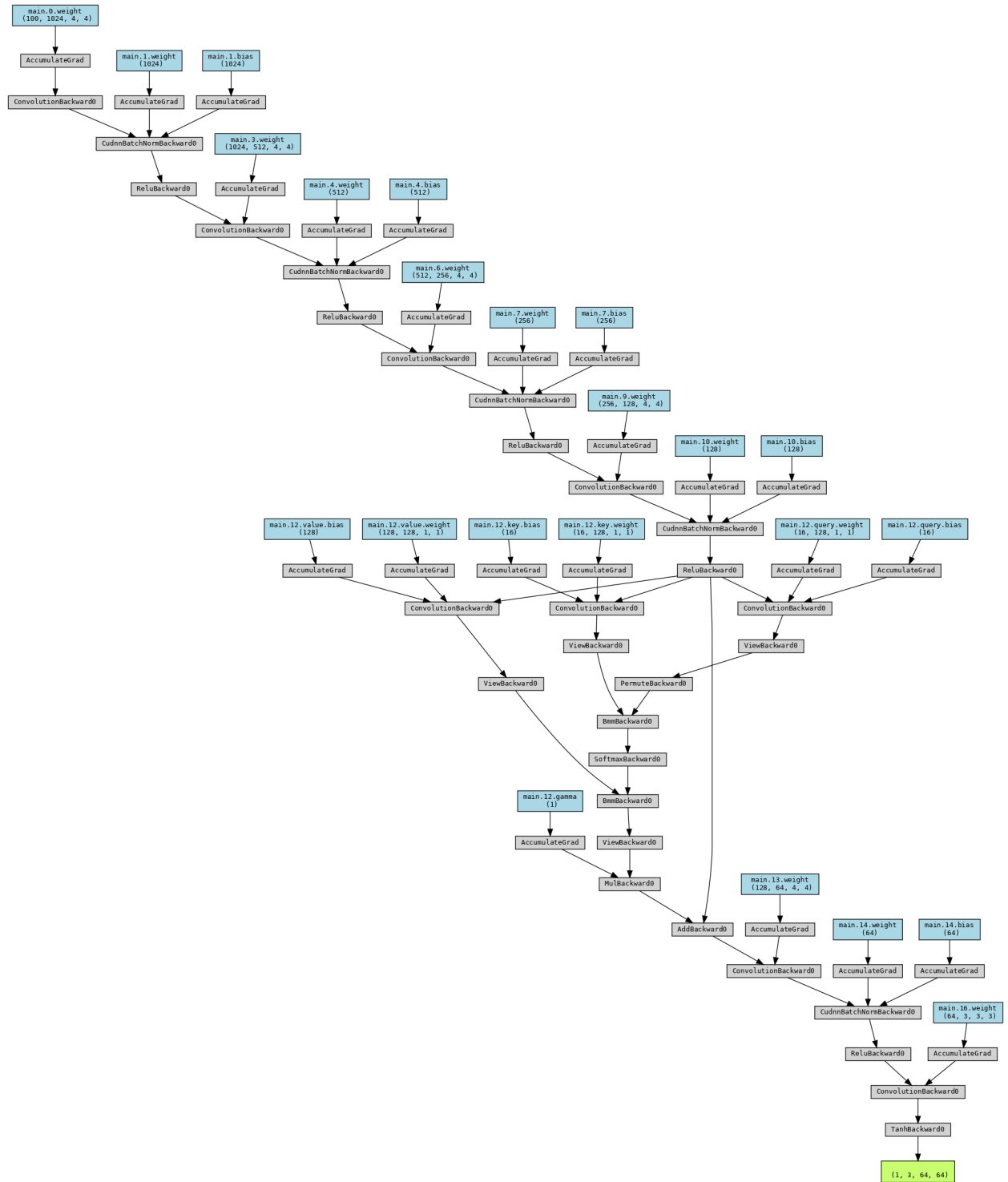
https://colab.research.google.com/drive/1X4EA-FduKd_nMBnO54q3Vgxbl-fH72LS?usp=sharing

Experimental results

netD



netG



```

Generator(
  (main): Sequential(
    (0): ConvTranspose2d(100, 1024, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): ConvTranspose2d(1024, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (4): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU(inplace=True)
    (6): ConvTranspose2d(512, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (7): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (8): ReLU(inplace=True)
    (9): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (10): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (11): ReLU(inplace=True)
    (12): SelfAttn(
      (query): Conv2d(128, 16, kernel_size=(1, 1), stride=(1, 1))
      (key): Conv2d(128, 16, kernel_size=(1, 1), stride=(1, 1))
      (value): Conv2d(128, 128, kernel_size=(1, 1), stride=(1, 1))
    )
    (13): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (14): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (15): ReLU(inplace=True)
    (16): ConvTranspose2d(64, 3, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (17): Tanh()
  )
)
Discriminator(
  (main): Sequential(
    (0): Conv2d(3, 64, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (1): LeakyReLU(negative_slope=0.2, inplace=True)
    (2): Conv2d(64, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (3): LeakyReLU(negative_slope=0.2, inplace=True)
    (4): Conv2d(128, 256, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (5): LeakyReLU(negative_slope=0.2, inplace=True)
    (6): SelfAttn(
      (query): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1))
      (key): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1))
      (value): Conv2d(256, 256, kernel_size=(1, 1), stride=(1, 1))
    )
    (7): Conv2d(256, 512, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1), bias=False)
    (8): LeakyReLU(negative_slope=0.2, inplace=True)
    (9): Conv2d(512, 1, kernel_size=(4, 4), stride=(1, 1), bias=False)
    (10): Sigmoid()
  )
)

```

本次架構調整說明

1. Generator

- 通道倍增至 `ngf×16`，提升生成器容量
- 在 32×32 特徵圖加入 **Self-Attention** → 改善長距離一致性
- 最後改用 3×3 卷積輸出 (減少棋盤效應)

2. Discriminator

- 所有卷積層加 **Spectral Normalization** → 讓 Lipschitz 常數受控，收斂更平穩

- 與 G 相同的位置插入 Self-Attention
- 輸出 flatten 成向量，方便改成 WGAN-GP 時去掉 Sigmoid

3. 訓練設定

- 其餘超參 (lr, beta1, epochs ...) 與原 DCGAN 相同；僅把 netG, netD 指向新類別並 apply(weights_init) .



面向	優點	缺點
細節	加入注意力後，頭髮邊緣、背景花紋的 連續性明顯改善	極少數圖片仍出現 checkerboard artifacts
多樣性	通道數增加 → latent space 有更足自由度， 表情 / 姿勢多樣	容易出現「背景均一顏色」的 mode
穩定性	Spectral Norm 使判別器梯度變平滑， 訓練不再出現早期崩潰	收斂稍慢（每 epoch 多約 8 % 計算量）
訓練成本	參數量 ↑ 1.8×，VRAM 使用增加 ≈ 600 MB	若 GPU 只有 4 GB 需把 batch_size 從 128 降到 64

Conclusion

改良後的 DCGAN 在 **細節保真度** 與 **生成多樣性** 上皆優於原版：注意力機制讓長距離紋理更連續，Spectral Norm 使訓練過程更穩定，顯著減少早期崩潰與人臉扭曲；整體視覺品質提升，但代價是參數量與 VRAM 使用量增加，收斂速度稍慢。

Discussion

模型變大後對顯存挑剔；畫質雖提升，仍偶有棋盤格與背景單色，可靠資料增強或 StyleGAN-式 skip 進一步修補。